# Training-Free Multiview Semantic Segmentation of 3D Gaussian Splatting Scenes

Anubha Vyasamudri (av3329) Sanjita Chandan Ballapur (sb5216)

*COMS 4732W Computer Vision*

**Abstract** 3D Gaussian Splatting achieves photorealistic novel view synthesis in real time but provides no semantic understanding of scene content. We present a training free pipeline that segments individual objects in a pretrained Gaussian scene using text prompts, then applies downstream edits including recoloring, removal, and natural language instruction editing. The core contribution is multiview opacity-weighted mask lifting: 2D segmentation masks from GroundingDINO and SAM2 are fused across $N$=8 orbital views, with per-Gaussian scores accumulated via opacity weighting and refined through spatial post-filters. We demonstrate 3D-consistent recoloring on the Mip-NeRF 360 bonsai and bicycle scenes using gsplat for differentiable Gaussian rasterisation.

## INTRODUCTION

3D Gaussian Splatting (3DGS) [1] represents a scene as millions of anisotropic 3D Gaussians, each with a 3D position, covariance matrix, opacity, and spherical harmonic color coefficients. It renders photorealistic novel views at real time frame rates, substantially outperforming NeRF based methods.

Despite this representational power, 3DGS scenes are semantically blind. No Gaussian carries a label; there is no notion of which Gaussians belong to the tree, the bicycle, or the cloth backdrop. Methods that add semantic understanding either require retraining the scene from scratch [2, 3] or embedding features during the original optimisation [4]. We instead operate on *any* pretrained scene without modifying its Gaussian parameters, using only a natural language text prompt at inference time.

## PROBLEM FORMULATION

Given a pretrained 3DGS scene (a set of $N$ Gaussians with known parameters) and a text prompt $p$, produce a binary label $\mathbf{s} \in \{0,1\}^N$ indicating object membership for each Gaussian. This label is used for three downstream edits: (1) recoloring, (2) removal, and (3) natural language instruction editing. No training images or scene reoptimisation are required. The entire pipeline runs on a single T4 GPU in under 30 minutes.

## METHOD

### 3.1 Scene Representation and Data

We use pretrained '.splat' binary files from the publicly available `dylanebert/3dgs` HuggingFace dataset, which packages Mip-NeRF 360 scenes in a compact 32-byte-per-Gaussian format storing position (float32×3), scale (float32×3), RGBA (uint8×4), and quaternion rotation (uint8×4). We evaluate on the bonsai (272,956 Gaussians) and bicycle (567,714 Gaussians) scenes.

### 3.2 Orbit Camera Generation

Rather than relying on the original COLMAP training cameras, we generate a synthetic orbital camera ring around the scene. The scene centre and orbit radius are estimated from the top-30% most opaque Gaussians. Cameras are placed on a single horizontal orbit at $20^\circ$ elevation with a $70^\circ$ horizontal field of view, using an OpenCV convention viewmatrix (Y-down, Z-forward). We use $N$=8 cameras for mask lifting and $N$=20 cameras for the output GIF. Each view is rendered at $480 \times 640$ pixels using gsplat rasterisation [8] with a white background.

### 3.3 2D Segmentation per View

Each rendered view is passed to a two-stage 2D segmentation pipeline:

- **GroundingDINO** [6] localises the target object as one or more bounding boxes given the text prompt. We use detection threshold $\tau_{\text{box}}$=0.28 and text threshold $\tau_{\text{txt}}$=0.25.
- **SAM2** [5] refines each box into a pixel-accurate binary mask using its image predictor.

If GroundingDINO detects nothing in a given view the mask for that view is empty and contributes no votes during lifting.

### 3.4 Opacity-Weighted Multiview Lifting

The core of the pipeline fuses per-view 2D masks into a 3D Gaussian segment via opacity-weighted projection. For each view $v$ with viewmatrix $\mathbf{V}_v$ and intrinsics $\mathbf{K}_v$:

1. Project each Gaussian centre $\boldsymbol{\mu}_i$ to pixel coordinates $(u_i^v, v_i^v)$ via perspective projection.
2. Look up the mask value $m_i^v \in \{0,1\}$ at that pixel.
3. Accumulate an opacity-weighted vote:

$$s_i \mathrel{+}= \alpha_i \cdot m_i^v \tag{1}$$

where $\alpha_i \in [0,1]$ is the Gaussian's stored opacity.

After processing all $V$ views, $s_i$ is the total opacity-weighted evidence that Gaussian $i$ belongs to the target object. Gaussians that project outside the image frustum contribute zero.

### 3.5 Thresholding and Spatial Filters

**Score threshold.** Let $\mathcal{S}^+ = \{s_i : s_i > 0\}$ be the non-zero scores. We apply the 70th percentile threshold:

$$\hat{s} = \mathrm{p}_{70}(\mathcal{S}^+) \tag{2}$$

retaining all Gaussians with $s_i \geq \hat{s}$. This keeps the top 30% of activated Gaussians and discards the long tail of weakly-activated background Gaussians.

**Spatial post-filters.** The score threshold alone is insufficient because the lifting projection is ambiguous: background Gaus-

sians behind the object project into the same masked pixels and accumulate votes. We apply two spatial filters exploiting the Y-down world convention of the .splat files (smaller Y = physically higher):

- *Y-axis cutoff*: retain only Gaussians with $y < Y_{\text{max}}$, where $Y_{\text{max}}$ is the 35th percentile of the Y-coordinates of score-passing Gaussians, clamped to $[-0.2, 0.5]$. This separates the bonsai canopy and trunk (small Y, physically high) from the draped cloth background (large Y, physically low).
- *XZ-radius filter (bonsai only)*: retain only Gaussians within 0.55 world units of the scene XZ-centre, constraining the segment to the narrow trunk and canopy footprint and excluding the widely-spread cloth.
- *Brightness filter (bicycle only)*: retain only Gaussians with mean RGB $> 0.28$, exploiting the fact that the bicycle frame is silver/white whereas the park bench is black.

### 3.6 Downstream Editing Operations

Given the binary segment $\mathbf{s}$, three editing operations are supported, all with $O(N)$ complexity:

**Recolor.** Replace the color of each segmented Gaussian with a target RGB value: $\mathbf{c}_i \leftarrow \mathbf{c}_{\text{target}}$ for all $i$ with $s_i = 1$.

**Remove.** Render the scene using only the complement $\{i : s_i = 0\}$, effectively deleting the segmented object.

**Natural language instruction editing.** A runtime regex parser extracts an object noun and a color name from instructions of the form *"change the X to Y"*. The object token is passed directly to GroundingDINO as a new prompt; the color token is mapped to a target RGB value. The full segmentation and lifting pipeline then re-runs on the new prompt without re-rendering the scene.

## RESULTS

### 4.1 Bonsai: Recolor and Remove

Using the prompt *potted plant* on the bonsai scene (272,956 Gaussians), the pipeline selected 9,501 Gaussians (3.5%) corresponding to the tree trunk, branches, and pot. The recoloring to red is 3D-consistent across all 20 orbit frames: the same set of Gaussians is recolored regardless of viewpoint, producing a globally coherent edit that a 2D per-frame method could not achieve.




**Figure 1:** Bonsai scene. Original (top) and recolored red (bottom). Prompt: *potted plant*. 9,501 of 272,956 Gaussians.

### 4.2 Bicycle: Recolor and Remove

The bicycle scene (567,714 Gaussians) is more challenging: the bicycle and the park bench occupy a similar depth plane and partial overlap in the 2D projections. The brightness filter successfully separates the silver bicycle from the black bench. The pipeline selected 65,144 Gaussians (11.5%) and recolored the bicycle blue across all orbit frames while leaving the bench, grass, and path unchanged.




**Figure 2:** Bicycle scene. Original (top) and recolored blue (bottom). 65,144 of 567,714 Gaussians recolored.

### 4.3 Natural Language Instruction Editing

A runtime parser accepts instructions such as *change the flowers to green*. The object token *flowers* feeds GroundingDINO as a new segmentation prompt; the color token *green* maps to RGB $(0.10, 0.80, 0.15)$. The same lifting pipeline runs from scratch on the new prompt; only the 2D segmentation and score accumulation steps need to repeat. Figure 3 shows the result on the bonsai scene.



**Figure 3:** NL instruction edit: *change the flowers to green*.

## ANALYSIS

**Strengths.**

- All edits operate directly on the 3D Gaussian set, so 3D consistency across viewpoints is guaranteed by construction without any per-frame postprocessing.
- The pipeline is entirely training free and runs on any pretrained '.splat' file without requiring original training images.
- The modular design (render $\rightarrow$ segment $\rightarrow$ lift $\rightarrow$ edit) supports all three editing modes from a single segmentation pass.

**Limitations.**

- *Occluded Gaussian contamination.* Naive projection assigns votes to every Gaussian projecting into a masked pixel, including those physically behind the visible surface. Interior Gaussians inside the bonsai trunk, for example, accumulate the same score as surface Gaussians, causing over-segmentation.

- *Scene-specific spatial filters.* The Y-axis cutoff and XZ-radius thresholds are tuned per scene by inspecting the Gaussian coordinate distribution. They do not generalise automatically to new scenes.
- *Single-elevation camera ring.* Using cameras at a fixed $20^\circ$ elevation means that the top of the bonsai canopy and the underside of the bicycle are underrepresented in the lift views, leading to lower lifting scores for those surface regions.
- *Fixed percentile threshold.* The p70 threshold is set heuristically and does not adapt to the actual score distribution. Scenes with sparse or uneven activation require manual adjustment.
- *No quantitative evaluation.* mIoU against ground-truth segmentation was not computed because the '.splat' files do not include per-Gaussian semantic labels.

**What we would do differently.** Rendering a per-view depth map and gating votes by surface depth would eliminate occluded Gaussian contamination without any additional annotation. Replacing the fixed percentile threshold with Otsu's method would find the natural foreground/background split in score space automatically. Using three elevation rings ($25^\circ$, $5^\circ$, $-10^\circ$) instead of one would provide complete coverage of object surfaces. Eroding the 2D masks before lifting by a few pixels would remove boundary-ambiguous votes that cause halo artifacts at the segment edge.

## FUTURE WORK

- **Depth-aware lifting.** Render a Z-buffer per view and only award a vote when the Gaussian's camera-space depth matches the surface depth within a relative tolerance. This directly addresses the primary failure mode of occluded Gaussian contamination.
- **Feature field lifting.** Assign CLIP or DINOv2 features to each Gaussian via multiview averaging, enabling open-vocabulary segmentation without the bounding-box bottleneck of GroundingDINO.
- **Style transfer.** Optimise spherical harmonic coefficients of segmented Gaussians against a style image for appearance and texture transfer, rather than flat uniform recoloring.
- **Video extension.** Apply the same lifting approach to 4D Gaussian scenes for temporally consistent object tracking and editing in video.

## REFERENCES


[1] B. Kerbl et al. 3D Gaussian splatting for real-time radiance field rendering. *ACM SIGGRAPH*, 2023.

[2] M. Ye et al. Gaussian grouping: Segment and edit anything in 3D scenes. *ECCV*, 2024.

[3] J. Liu et al. LERF: Language embedded radiance fields. *ICCV*, 2023.

[4] Y. Shi et al. Language embedded 3D Gaussians for open-vocabulary scene understanding. *CVPR*, 2024.

[5] N. Ravi et al. SAM 2: Segment anything in images and videos. *Meta AI*, 2024.

[6] S. Liu et al. Grounding DINO: Marrying DINO with grounded pre-training. *arXiv*, 2023.

[7] J. Barron et al. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022.

[8] V. Ye et al. gsplat: An open-source library for Gaussian splatting. *arXiv*, 2024.